MarketSight

Stock Market Prediction System

Sprint 4 - Construction 2

Development Report

Team Members: Zane Tessmer (Scrum Master) Harsh Kumar (Product Owner / Lead Developer) Dante Bertolutti (Developer) Parag Jindal (Developer) Kaval S (Developer) Guntash (Developer)

Date: **March 3, 2026** Course: COSC 471 - Winter 2026 Okanagan College

Table of Contents

1. Executive Summary
2. Team Roles & XP Pair Assignments
3. Sprint Overview & Timeline
4. Work Completed by Dante Bertolutti
5. Work Completed by Harsh Kumar
6. System Architecture Overview
7. Backend API Endpoints
8. Frontend Pages & Components
9. Machine Learning Pipeline
10. Database Schema
11. CI/CD Pipeline
12. Docker Infrastructure
13. Testing Summary
14. Commit Log (Dante & Harsh)
15. Configuration Management
16. Known Issues & Next Steps

1. Executive Summary

This report consolidates all development work completed by **Dante Bertolutti** and **Harsh Kumar** during Sprint 4 (Construction 2) of the MarketSight project. MarketSight is a real-time ML-powered stock price prediction platform built with FastAPI, React 19, XGBoost, and PostgreSQL.

During this sprint, the team focused on building the prediction confidence scoring system, the predictions UI page, Airflow DAG automation, Nginx reverse proxy configuration, comprehensive backend testing, frontend ML integration, and CI/CD pipeline improvements. The system is now capable of end-to-end stock price prediction with confidence scores, automated data seeding, and weekly model retraining.

Key metrics: 10 backend test files, 6 CI jobs, 7 Docker services, 3 Airflow DAGs, 5 API endpoints, 8 frontend pages, and a 17-feature XGBoost model with heuristic confidence scoring.

2. Team Roles & XP Pair Assignments

Team Member	Role
Zane Tessmer (Foochini)	Scrum Master
Harsh Kumar (Harshksaw)	Product Owner / Lead Developer
Dante Bertolutti	Developer (XP Pair with Harsh)
Parag Jindal (Paragjindal01)	Developer
Kaval S (KavalS)	Developer
Guntash (guntash499)	Developer

3. Sprint Overview & Timeline

Sprint Duration: February 10 - March 3, 2026 (3 weeks)

Sprint Goal: Deliver a functional MVP with end-to-end prediction capability, automated data pipelines, and comprehensive testing.

Sprint Timeline

Date	Developer	Work Completed
Feb 4-9	Harsh	Initial project scaffolding, ML notebook, DB setup, Docker configs
Feb 10	Harsh	Frontend redesign (MarketSight branding), market module, CI/CD pipeline
Feb 10	Harsh	Backend migration: SQLAlchemy ORM, uv package manager, Docker consolidation
Feb 11	Harsh	Inference module: API endpoint, service, model loading, feature engineering
Feb 11	Harsh	Frontend ML integration: prediction display, chart overlays, dashboard UI
Feb 11	Harsh	Testing: unit tests, integration tests, backend linting, CI fixes
Feb 11	Harsh	Airflow DAGs: data seeding, model retraining automation
Feb 17	Harsh	Nginx proxy config, DRI documentation, LightGBM migration
Feb 23	Dante	Prediction confidence scoring system (backend + tests)
Feb 25	Dante	Predictions page UI, confidence display, frontend type alignment

4. Work Completed by Dante Bertolutti

4.1 Prediction Confidence Scoring System (Backend)

Commit: `76485dc` | **Date:** Feb 23, 2026

Designed and implemented a heuristic confidence scoring system for stock price predictions. The system calculates a confidence score between 0.0 and 1.0 by combining four weighted factors:

- **Volatility Factor (35% weight):** Low recent volatility yields higher confidence using hyperbolic decay.
- **Return Magnitude Factor (30% weight):** Smaller predicted returns yield higher confidence via Gaussian decay.
- **Volume Ratio Factor (20% weight):** Normal trading volume (ratio near 1.0) yields higher confidence.
- **RSI Factor (15% weight):** RSI in the 40-60 neutral range yields maximum confidence; extreme values reduce it.

Files created/modified:

- `backend/app/modules/inference/confidence.py` (138 lines) - Core confidence calculation module
- `backend/app/modules/inference/service.py` - Integrated confidence into prediction pipeline
- `backend/tests/modules/test_confidence.py` (213 lines) - Comprehensive unit tests
- `backend/tests/modules/test_inference.py` (60+ lines added) - Updated inference integration tests

Total: 412 insertions, 5 deletions across 4 files.

4.2 Predictions Page UI (Frontend)

Commit: `9036d57` | **Date:** Feb 25, 2026

Built the full Predictions page in the frontend dashboard. Users can select individual stocks or use "Predict All" to generate predictions for every active stock. Each prediction is displayed as

a card showing the stock symbol, current price, predicted price, predicted return percentage, and a confidence score with color-coded indicators (green for high, yellow for medium, red for low).

Files created/modified:

- `frontend/src/routes/dashboard/predictions.tsx` (240+ lines rewritten) - Full predictions page
- `frontend/src/hooks/useInference.ts` - Updated query hooks for prediction data
- `frontend/src/services/inference.ts` - Aligned API client with backend response schema
- `frontend/src/types/index.ts` - Updated TypeScript type exports
- `frontend/src/types/inference.ts` - Removed (consolidated into index.ts)
- `frontend/src/routes/dashboard/stocks.$symbol.tsx` - Updated prediction overlay on stock detail

Total: 260 insertions, 41 deletions across 6 files.

4.3 Code Review & Inspection

Reviewed Harsh's inference module implementation and Airflow DAG configurations. Provided feedback on error handling patterns in the prediction service and suggested the confidence scoring approach that was subsequently implemented.

5. Work Completed by Harsh Kumar

5.1 Project Foundation & Architecture (Feb 4-10)

- Established initial project structure: ML, backend, frontend modules with Dockerfiles
- Implemented XGBoost stock prediction notebook with automatic feature engineering
- Set up PostgreSQL via Docker Compose with data integration pipeline
- Trained global multi-stock prediction model (17 features, 500 estimators)
- Introduced classifier model training and inference testing scripts
- Migrated dependency management from pip to uv for faster builds
- Configured docker-compose with locally built images and explicit naming

5.2 Backend Development (Feb 10-17)

- Migrated to SQLAlchemy 2.0 ORM with Mapped type annotations
- Implemented market module: Stock and DailyPrice models, CRUD, service layer, API endpoints
- Built inference module: prediction API endpoint, model loading (singleton pattern), feature engineering
- Created seed data scripts for development and testing
- Configured Alembic migrations (6 migration files)
- Set up FastAPI with CORS, health checks, and modular router architecture
- Configured Nginx reverse proxy for backend access
- Removed legacy authentication modules and unused code

5.3 Frontend Development (Feb 10-11)

- Redesigned welcome page with new feature sections and MarketSight branding
- Built interactive StockChart component using TradingView Lightweight Charts
- Implemented stock detail pages with candlestick/line chart modes and time range selector
- Added dashed prediction line overlay on stock charts
- Integrated stock prediction display directly on stock symbol pages
- Refactored frontend services from static classes to functional modules

- Created dashboard UI with summary cards (Total Stocks, Sectors, Model Version, API Status)

5.4 Machine Learning Pipeline (Feb 4-17)

- Trained XGBoost model on historical data with 17 technical indicator features
- Implemented feature engineering: RSI, MACD, Bollinger Bands, SMA distances, volatility, volume ratios
- Created ModelManager singleton for thread-safe model loading and caching
- Explored LightGBM migration with dedicated training and data inspection scripts
- Built global model training script with TimeSeriesSplit cross-validation
- Achieved Test RMSE: 0.1140, Test MAE: 0.0918

5.5 Airflow DAGs & Automation (Feb 11-17)

- **seed_market_data DAG:** Daily at 1:00 AM - truncates old data, seeds fresh stock prices, verifies data quality
- **retrain_model DAG:** Weekly Sundays at 2:00 AM - validates data freshness, retrains XGBoost, evaluates, backs up artifacts
- **hello_airflow DAG:** Connectivity testing
- Configured Airflow services in Docker Compose (webserver, scheduler, init)

5.6 Testing & CI/CD (Feb 11)

- Wrote unit tests for market module CRUD operations
- Wrote integration tests for market API endpoints and inference API
- Wrote service layer tests for MarketService
- Set up conftest.py with DB session fixtures and test data seeding
- Created CI pipeline: backend-lint, backend-typecheck, backend-test, frontend-lint, frontend-build, docker-build
- Fixed CI errors: schema dropping, Alembic migrations, import ordering, linting

5.7 Documentation

- Refactored project README with updated features, architecture, and tech stack
- Created backend-specific README

- Wrote DRI documentation for JupyterHub/HPC server access
- Added development guide with code quality commands

6. System Architecture Overview

MarketSight follows a three-tier architecture with clear separation between the presentation layer (React frontend), business logic layer (FastAPI backend), and data layer (PostgreSQL + ML models).

Architecture Components

- **Frontend:** React 19 + TypeScript + Vite, TanStack Router (file-based routing), TanStack Query, TailwindCSS, shadcn/ui, TradingView Lightweight Charts
- **Backend:** FastAPI (async), SQLAlchemy 2.0, Alembic migrations, Pydantic v2 validation
- **ML Engine:** XGBoost Regressor with thread-safe singleton ModelManager
- **Database:** PostgreSQL 16 with `market` schema (stocks + daily_prices tables)
- **Automation:** Apache Airflow 2.10.4 (data seeding + model retraining DAGs)
- **Infrastructure:** Docker Compose with 7 services, Nginx reverse proxy
- **CI/CD:** GitHub Actions with 6 jobs (lint, typecheck, test, build)

Data Flow

1. **Ingestion:** Airflow seed_market_data DAG populates market.stocks and market.daily_prices daily
2. **Processing:** Feature engineering pipeline calculates 17 technical indicators from OHLCV data
3. **Inference:** XGBoost model predicts next closing price with confidence score
4. **Persistence:** Predictions served via REST API (future: stored for accuracy tracking)
5. **Consumption:** React frontend fetches data via TanStack Query with configurable stale times

7. Backend API Endpoints

Base URL: `/api/v1` | **Framework:** FastAPI | **Port:** 8000

Method	Route	Description
GET	`/market/stocks`	List all active stocks (for dropdowns/tables)
GET	`/market/stocks/{symbol}`	Get metadata for a single stock by ticker

GET	`/market/stocks/{symbol}/ohlc`	Get daily OHLC + volume data (query: days=365)
GET	`/inference/predict/{symbol}`	Get 1-day price prediction with confidence score
GET	`/utils/health-check/`	Returns true if service is healthy

8. Frontend Pages & Components

Pages (Routes)

URL Path	Description
`/`	Root landing / welcome page with feature sections
`/dashboard`	Dashboard layout wrapper with sidebar navigation
`/dashboard/`	Main dashboard with summary cards (stocks, sectors, model, API status)
`/dashboard/stocks`	Stocks list with search and sector-based tab filtering
`/dashboard/stocks/:symbol`	Stock detail: OHLC chart (candlestick/line), prediction overlay, metadata
`/dashboard/predictions`	Predictions page: select stock or Predict All, confidence cards
`/dashboard/admin`	Admin: user management (superuser-only)
`/dashboard/settings`	User settings: profile, password, account deletion

Key Components

- **StockChart.tsx** - TradingView Lightweight Charts with candlestick/line modes and prediction line
- **DataTable.tsx** - Reusable sortable/filterable data table component
- **AppSidebar.tsx** - Application sidebar navigation
- **AuthLayout.tsx** - Authentication wrapper layout
- **Admin components:** AddUser, EditUser, DeleteUser, UserActionsMenu
- **UserSettings components:** ChangePassword, DeleteAccount, UserInformation

9. Machine Learning Pipeline

Model Details

Parameter	Value
Model Type	XGBoost Regressor (global multi-stock)
Training Date	February 11, 2026
Prediction Horizon	26 periods
n_estimators / max_depth	500 / 6
learning_rate / subsample	0.05 / 0.8
Test RMSE / MAE	0.1140 / 0.0918
Feature Count	17
Cross-Validation	TimeSeriesSplit

Feature Categories (17 Features)

- **Lagged Returns:** ret_1, ret_5, ret_10, ret_20
- **SMA Distance:** dist_sma_10, dist_sma_20, dist_sma_50
- **Momentum:** rsi, macd, macd_signal, macd_diff
- **Bollinger Bands:** bb_width, bb_position
- **Volatility:** volatility_20
- **Volume:** vol_ratio
- **Time:** dayofweek, month

Confidence Scoring (Implemented by Dante)

- **Volatility factor (35%):** Hyperbolic decay - low volatility = high confidence
- **Return magnitude factor (30%):** Gaussian decay - small predictions = high confidence
- **Volume ratio factor (20%):** Gaussian centered at 1.0 - normal volume = high confidence
- **RSI factor (15%):** Piecewise linear - RSI 40-60 = max confidence, extremes reduce it

10. Database Schema

Database: PostgreSQL 16 | **ORM:** SQLAlchemy 2.0 | **Schema:** market | **Migrations:** Alembic

market.stocks (Stock Dimension Table)

Column	Type	Notes
symbol	String (PK)	Ticker symbol, e.g. 'AAPL'
name	String	Full company name
sector	String (nullable)	e.g. 'Technology'
industry	String (nullable)	e.g. 'Consumer Electronics'
currency	String	Default 'USD'
exchange	String (nullable)	e.g. 'NASDAQ'
is_active	Boolean	Default True

market.daily_prices (OHLC Fact Table)

Column	Type	Notes
id	Integer (PK)	Auto-increment, indexed
symbol	String (FK)	References market.stocks.symbol
date	Date	Trading date, indexed
open / high / low / close	Float	OHLC price data
volume	Integer	Trading volume
previous_close	Float (nullable)	Prior day close
change / change_pct	Float (nullable)	Price change and percentage

Unique constraint on (symbol, date). Migration history: 6 Alembic revisions from initial models to market module with cascade deletes.

11. CI/CD Pipeline

Platform: GitHub Actions | **Triggers:** Push/PR to main branch

ci.yml - Main Pipeline (6 Jobs)

Job	Description
backend-lint	Install deps with uv, run `ruff check` and `ruff format --check`

backend-typecheck	Run `mypy app` (strict mode)
backend-test	Spin up PostgreSQL 16 service, run `alembic upgrade head`, then `pytest tests/ -v`
frontend-lint	Install with bun, run `bunx biome check`
frontend-build	Build frontend with `bun run build` (sets VITE_API_URL)
docker-build	Matrix build for backend and frontend Dockerfiles with GHA cache

backend-tests.yml - Backend-Only Tests

Triggers on push to `backend/**` files. Runs PostgreSQL service + Alembic + pytest for fast feedback.

12. Docker Infrastructure

Orchestration: docker-compose.yml | **7 Services**

Service	Image	Port	Purpose
db	postgres:16	5432	Primary PostgreSQL database
adminer	adminer	8080	Database web UI
prestart	backend Dockerfile	-	One-time migrations + seed
backend	backend Dockerfile	8000	FastAPI app (4 workers)
frontend	frontend Dockerfile	5173	Nginx-served React app
airflow-web	airflow:2.10.4	8081	Airflow UI (admin/admin)
airflow-sched	airflow:2.10.4	-	DAG scheduler

13. Testing Summary

Test Files & Coverage

Test File	What It Tests
test_utils.py	Health check endpoint returns true with 200 status
test_market.py	Market API: list stocks, get by symbol, 404s, OHLC structure
test_market_crud.py	DB CRUD: active stocks ordering, None for missing, date filtering
test_market_service.py	Service layer: StockRead objects, Unix timestamps, empty ranges
test_inference.py	Inference API: 404 for unknown, mocked pipeline, confidence validation
test_confidence.py	Confidence scoring: all sub-functions, ranges, monotonic, NaN, extremes
conftest.py	Shared fixtures: db session, FastAPI TestClient with DB override
modules/conftest.py	Market data fixture: seeds 4 stocks + 5 days OHLC for AAPL

Test Categories

- **Unit Tests:** test_confidence.py (all confidence sub-functions), test_market_crud.py (DB queries)
- **Integration Tests:** test_market.py, test_inference.py (full API request/response cycle)
- **Regression Tests:** CI pipeline re-runs all tests on every push/PR to main
- **Acceptance Tests:** Manual verification of prediction accuracy and UI behavior

14. Complete Commit Log (Dante & Harsh)

Dante Bertolutti - Commits

Hash	Date	Message
9036d57	Feb 25	feat: Build predictions page, add confidence display, align frontend types with backend schemas
76485dc	Feb 23	feat: Add prediction confidence scores to inference API

Harsh Kumar - Commits (Selected Key Commits)

Hash	Date	Message

4b48263	Feb 17	refactor: Migrate stock prediction to LightGBM with dedicated training scripts
ef72646	Feb 17	feat: Enable backend access by configuring Nginx proxying and DRI docs
8d6c7e9	Feb 11	feat: Add initial Airflow DAGs and update Docker configurations
5963e8b	Feb 11	chore: Install curl in backend Dockerfile, add .gitkeep files for Airflow
1f371c5	Feb 11	refactor: Refactor frontend services from static classes to functional modules
0f8487b	Feb 11	test: Add tests for market CRUD, service, API, and inference API
1a392a9	Feb 11	Unit test and Integration Tests
8de5ae5	Feb 11	feat: Integrate stock prediction display on stock symbol page
0afcc5c	Feb 11	feat: Add dashed prediction line to stock charts
d900314	Feb 11	feat: Implement stock prediction display and chart type selection
11fea12	Feb 11	feat: Implement inference module: API, service, model loading, features
63e344b	Feb 11	feat: Stock prediction model with technical indicators and training script
20ce7ed	Feb 11	feat: Rename project to MarketSight, update branding, add StockChart
9e58d7c	Feb 11	feat: Redesign welcome page with new feature sections
a003c91	Feb 10	refactor: Migrate market data from candles to daily prices and stock metadata
5cd0efc	Feb 10	feat: Migrate to SQLAlchemy ORM, remove auth, enable Docker Compose
97d25f8	Feb 10	feat: Add GitHub Actions CI workflow for full pipeline
7f270d9	Feb 10	feat: Migrate dependency management from pip to uv
57eb00b	Feb 4	feat: Establish initial project structure for ML, backend, and frontend
5b2001c	Feb 4	feat: Implement XGBoost stock prediction notebook with feature engineering

15. Configuration Management

Version Control

- **Repository:** GitHub (Okanagan-College-Cosc471-Winter-2026/the-project-maverick)
- **Branching Strategy:** Feature branches (ft-*) merged to main via Pull Requests
- **Current branch:** dante_feature (Dante's work)
- **Code Review:** PRs require review before merge

Environment Configuration

- **Backend:** .env file with PostgreSQL, Redis, model, polling, and API settings
- **Frontend:** VITE_API_URL and VITE_WS_URL environment variables
- **Docker:** docker-compose.yml with service-level environment variable injection
- **CI:** GitHub Actions secrets for database credentials in test jobs

Dependency Management

- **Backend:** uv (pyproject.toml + uv.lock) - Python 3.12
- **Frontend:** Bun (package.json + bun.lockb) - React 19, TypeScript 5
- **ML:** Conda (environment.yml) + pip (requirements.txt)
- **Docker:** Multi-stage builds with pinned base image versions

Key Configuration Files

File	Purpose
docker-compose.yml	Full stack orchestration (7 services)
backend/pyproject.toml	Python dependencies and tool config (ruff, mypy, pytest)
frontend/package.json	Frontend dependencies and scripts
backend/alembic.ini	Database migration configuration
.github/workflows/ci.yml	Main CI pipeline definition
.github/workflows/backend-tests.yml	Backend-only test pipeline
backend/app/core/config.py	Pydantic Settings (reads .env, validates secrets)

16. Known Issues & Next Steps

Known Issues

- Directional accuracy at 35.66% - model predicts magnitude better than direction
- WebSocket real-time streaming is planned but not yet implemented
- Frontend tests not yet written (test directories have .gitkeep placeholders)
- CORS configured to allow all origins (needs restriction for production)

Next Steps for Sprint 5

- Improve model accuracy with additional features and hyperparameter tuning
- Implement WebSocket streaming for real-time prediction updates
- Add frontend unit tests with Vitest
- Write acceptance test documentation with screenshots
- Implement prediction history storage and accuracy tracking
- Add user authentication back with JWT tokens
- Restrict CORS origins for production deployment